

GUÍA DE DEFENSA: HOSPITAL-VM

LogicTech | Presentación de Proyecto Fullstack

1. CRONOGRAMA DE LA EXPOSICIÓN (10 MINUTOS)

0 - 1 Min: Introducción y Contexto

Script:

"Buenos días. Somos LogicTech y hoy presentaremos Hospital-VM, una solución robusta para la gestión de pacientes, enfocada en escalabilidad y seguridad."

💡 Tip: Mantén contacto visual con el profesor. Establece el propósito del sistema de inmediato.

1 - 2.5 Min: Gestión y Planificación (EDT)

Script:

"Utilizamos una metodología estructurada. En la EDT cubrimos desde el análisis de requisitos hasta el QA. Actualmente, estamos en la fase de implementación final, asegurando la calidad mediante validaciones de Bean Validation."

2.5 - 5 Min: Arquitectura de Capas

Script:

"Nuestra arquitectura se basa en el Patrón de Capas para una separación de responsabilidades clara:"

Model: Entidades JPA con Lombok para limpieza de código.

Repository: Herencia de JpaRepository para CRUD eficiente.

Service: Aislamiento de la lógica de negocio pura.

Controller: Frontera REST y exposición de endpoints.

5 - 7 Min: Seguridad JWT

Script:

"Implementamos un flujo Stateless con JWT. El servidor emite un token que el cliente envía en el Header. El JwtFilter valida este acceso antes de procesar la petición en el controlador."

7 - 8.5 Min: Demo Técnica (Postman & Laragon)

Script:

"En Laragon gestionamos MySQL. Hibernate genera las tablas automáticamente. En Postman, un GET devuelve 200 OK y un POST devuelve 201 Created. Manejamos errores como el 401 si el token es inválido."

8.5 - 10 Min: Conclusión

Script:

"El desafío principal fue integrar seguridad sin afectar el rendimiento. Hospital-VM es una base sólida y escalable para entornos hospitalarios reales."

2. PREPARACIÓN PARA LA DEFENSA (PREGUNTAS PROBABLES)

¿Por qué usar Arquitectura de Capas y no todo en el Controller?

"Para facilitar el mantenimiento y las pruebas. Si cambiamos la base de datos o la lógica, solo modificamos una capa específica sin afectar la estabilidad de las demás."

¿Qué ventaja ofrece JWT sobre las sesiones tradicionales?

"JWT es stateless. El servidor no ocupa memoria para guardar sesiones, lo que permite que el sistema escale horizontalmente de forma mucho más sencilla y eficiente."

¿Cómo aseguran la integridad de los datos en la DB?

"Usamos Bean Validation en el modelo y restricciones JPA (como unique=true en el RUN) para validar los datos antes de que lleguen a la persistencia en MySQL."

3. VOCABULARIO TÉCNICO "SENIOR"

Boilerplate	Código repetitivo y genérico que evitamos mediante el uso de Lombok.
Stateless	Arquitectura donde el servidor no guarda información del estado del cliente (clave en JWT).
Inyección de Dependencias	Uso de @Autowired para desacoplar y conectar las capas del sistema.
Persistencia	El proceso de guardar datos de forma permanente en la base de datos MySQL.
Payload	La carga útil de datos que viaja en el cuerpo del JWT o de las peticiones POST.

4. RECOMENDACIONES PROFESIONALES

- **Gestión de Fallos:** Si la demo falla, explica la causa técnica (ej. puerto ocupado en Laragon) y describe el resultado esperado. Eso demuestra dominio.
- **Presencia:** Usa un tono de voz firme y formal. Evita muletillas como "ehh" o "bueno".
- **Visualización:** Si hablas del código en VS Code, no te detengas en cada línea; resalta las anotaciones clave (@Service, @Repository).